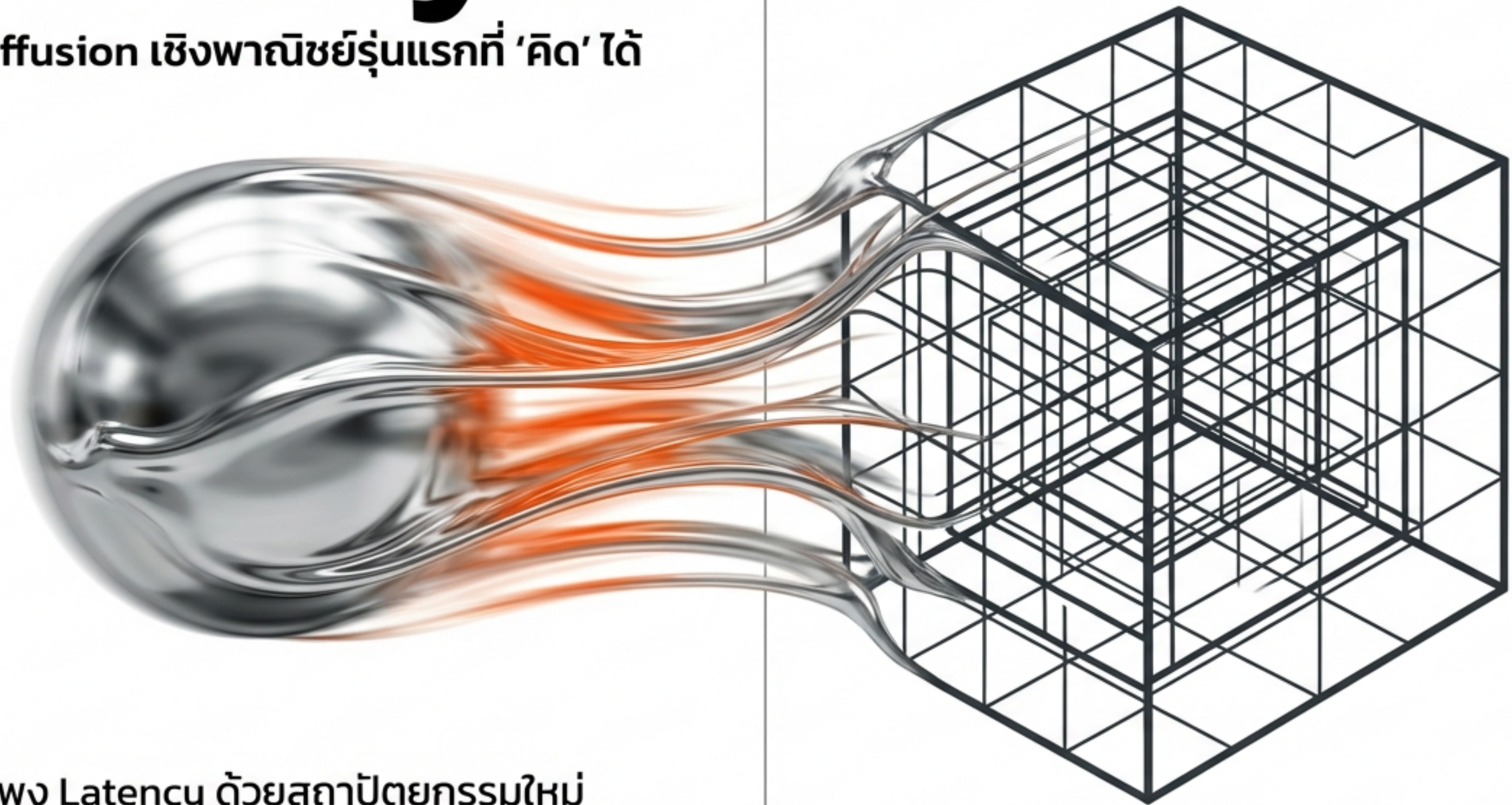


Mercury 2

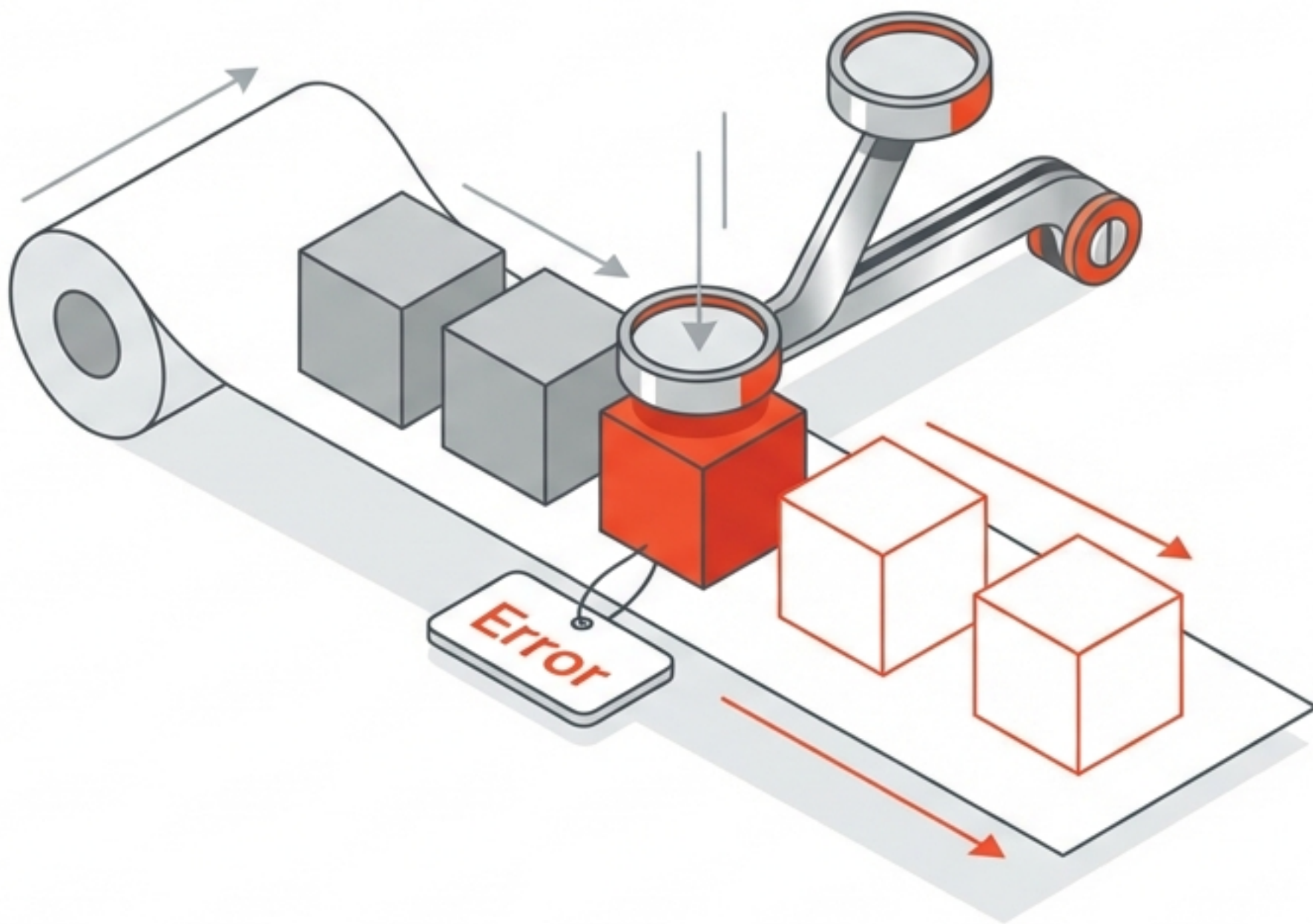
โมเดล Diffusion เชิงพาณิชย์รุ่นแรกที่ 'คิด' ได้



กลายกำแพง Latency ด้วยสถาปัตยกรรมใหม่
จาก Inception Labs

Sarabun
รายงานสรุปเชิงเทคนิค (Technical Review)

คอบวดของยุคปัจจุบัน: Auto-Regressive Models



Serial Decoding Latency: โมเดลปัจจุบันทำงานเหมือน 'เครื่องพิมพ์ดีด' (Typewriter) ที่ต้องสร้างข้อความทีละคำ เรียงต่อกันไปเรื่อยๆ

ข้อจำกัด: หากโมเดลทำผิดพลาดในขั้นตอนใดขั้นตอนหนึ่ง มันไม่สามารถย้อนกลับไปแก้ไขได้ (Permanent Mistakes)

ผลลัพธ์: ความเร็วในการประมวลผลถูกจำกัดด้วยลำดับการสร้าง (Sequential Process)

ทางออกใหม่: Diffusion Language Models



ความเร็วแสง ความเร็วแสง



Parallel Generation:

Diffusion เริ่มต้นจากสัญญาณรบกวน (Noise) และสร้างประโยคทั้งหมดขึ้นมา 'พร้อมกัน' ในคราวเดียว



The Text Editor Metaphor:

ทำงานเหมือนโปรแกรมแก้ไขข้อความที่สามารถแก้คำผิดตรงกลางประโยคได้ในขณะที่กำลังเขียนตอนท้าย



ประสิทธิภาพ:

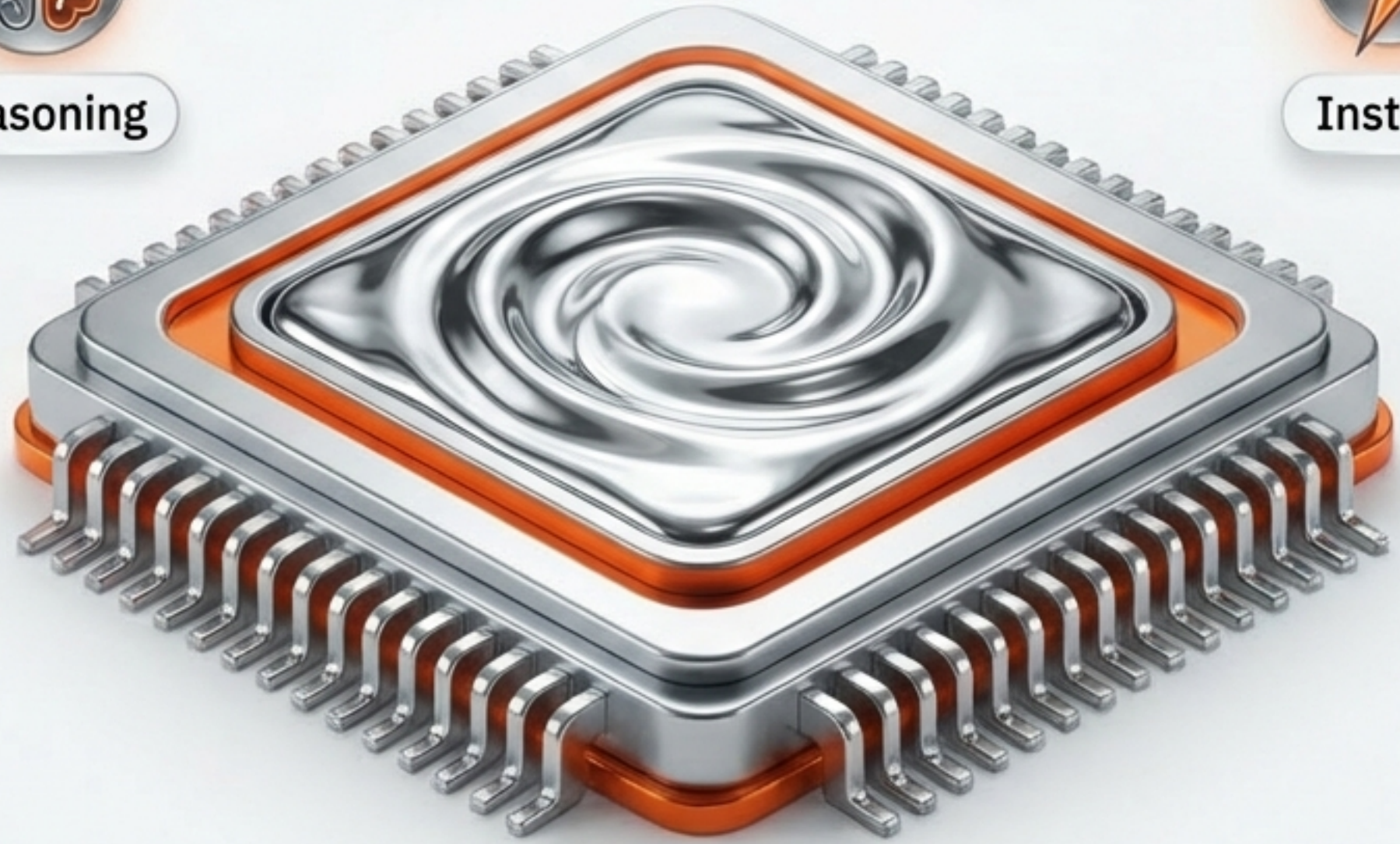
ให้ความเร็ว Inference speed สูงกว่าแบบเดิมถึง 10 เท่า

ขอแนะนำ Mercury 2

- **ผู้พัฒนา:** Inception Labs (ผู้สร้าง Mercury Coder)
- **นวัตกรรม:** ไม่ใช้การปรับแต่ง Hardware แต่เป็นการปฏิวัติที่ Algorithm
- **จุดเด่น:** เป็นโมเดล Diffusion ตัวแรกที่ยอมรับความสามารถด้าน **'Reasoning'** (การใช้เหตุผล) อย่างเต็มรูปแบบ



MERCURY 2



สองโหมดการทำงาน: ความเร็ว vs. การใช้เหตุผล



1. Instant Mode: เน้นความเร็วสูงสุด ปิดระบบการคิด
ซับซ้อน เหมาะสำหรับงานที่ต้องการผลลัพธ์ทันที
(Standard Diffusion)

2. High Reasoning Mode: เปิดใช้งานกระบวนการ
'คิด' (Iterative Refinement) เพื่อจัดการงานที่ซับซ้อน
แต่ยังคงรักษาความเร็วที่เหนือกว่าโมเดลทั่วไป

การทดสอบ: Coding & Instruction Following

`{ }` // โจทย์: เขียนเกม Tetris แบบย้อนกลับ (ตัวบล็อกลอยขึ้น) ในไฟล์ HTML เดียว

Mercury 2 - High Reasoning

```
1 <html>
2 <head>
3   <style></style>
4 </head>
5 <canvas width="300" user="thöck" wsc&kw"></canvas>
6 <script>
7   let board = { board };
8   let currentPiece = [new currentPiece'];
9   let currentPiece = "";
10  let board =
11
12  function game() {
13    if (surgame.loop(&edit) {
14      console.log(cavion on & rendeiss, currentPiece, board&);
15    }
16  }
17
18  function render() {
19    board.tackerstate(n + mast);
20    let currentPiece = [typica] currentPiece;
21    console.log(render(currentPiece));
```

**Finished**

GPT-5.2 - Instant

```
1 <!DOCTYPE html>
2 <html>
3 <head> |
```



Mercury 2 (High Reasoning): เข้าใจ Logic ที่ซับซ้อนและสร้างไฟล์เสร็จสิ้นในเสี้ยววินาที

GPT-5.2 Instant: ยังคงเขียนโค้ดทีละบรรทัด (Serial generation) ทำให้ใช้เวลานานกว่ามากสำหรับผลลัพธ์เดียวกัน

การควบคุมโครงสร้างภาษา (Structural Control)

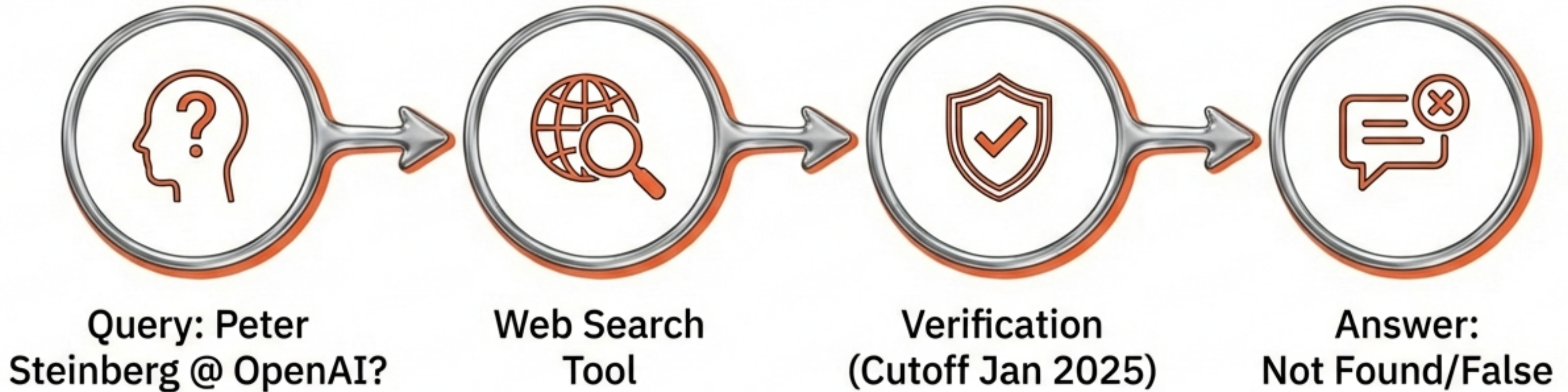
Instant Mode: ล้มเหลวใน
การนับคำที่แม่นยำ

High Reasoning Mode:
ทำได้สมบูรณ์แบบ
(Coherent & Accurate)
แสดงให้เห็นถึงความสามารถ
ในการวางแผนล่วงหน้าของ
Diffusion model ที่แก้ไข
ข้อความได้ทั้งประโยค



โจทย์: แต่งเรื่อง
ที่ประโยคถัดไป
ยาวขึ้นทีละ 1 คำ
(จาก 2 ถึง 20
คำ) แล้วลดลง
กลับมาที่ 2 คำ

ความสามารถด้าน Agent และ Web Search

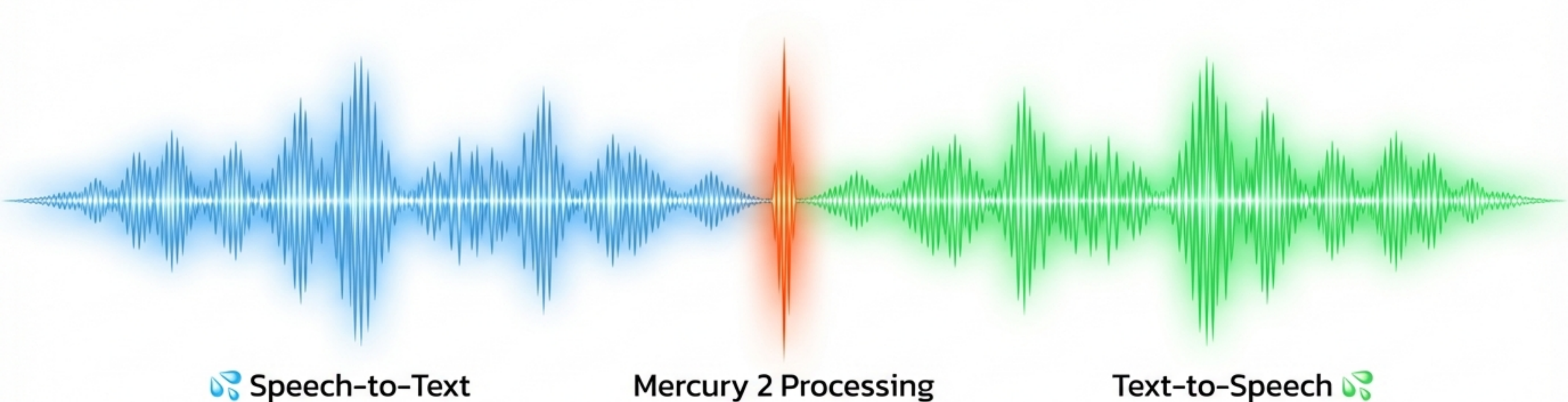


การตรวจสอบข้อเท็จจริง: เมื่อถามถึงข้อมูลเท็จ (เช่น การจ้าง Peter Steinberg ที่ไม่มีจริง) โมเดลสามารถค้นหาข้อมูลและยืนยันว่า 'ไม่มีเหตุการณ์นี้เกิดขึ้น'

Tool Use: สามารถใช้งานเครื่องมือภายนอกเพื่อตรวจสอบข้อมูลอัปเดต (Cutoff Date: มกราคม 2025) ได้อย่างแม่นยำ

Use Case: ผู้ช่วยเสียงแบบ Real-Time

Scenario: Customer Support สำหรับร้าน 'TechVault' 🎙️

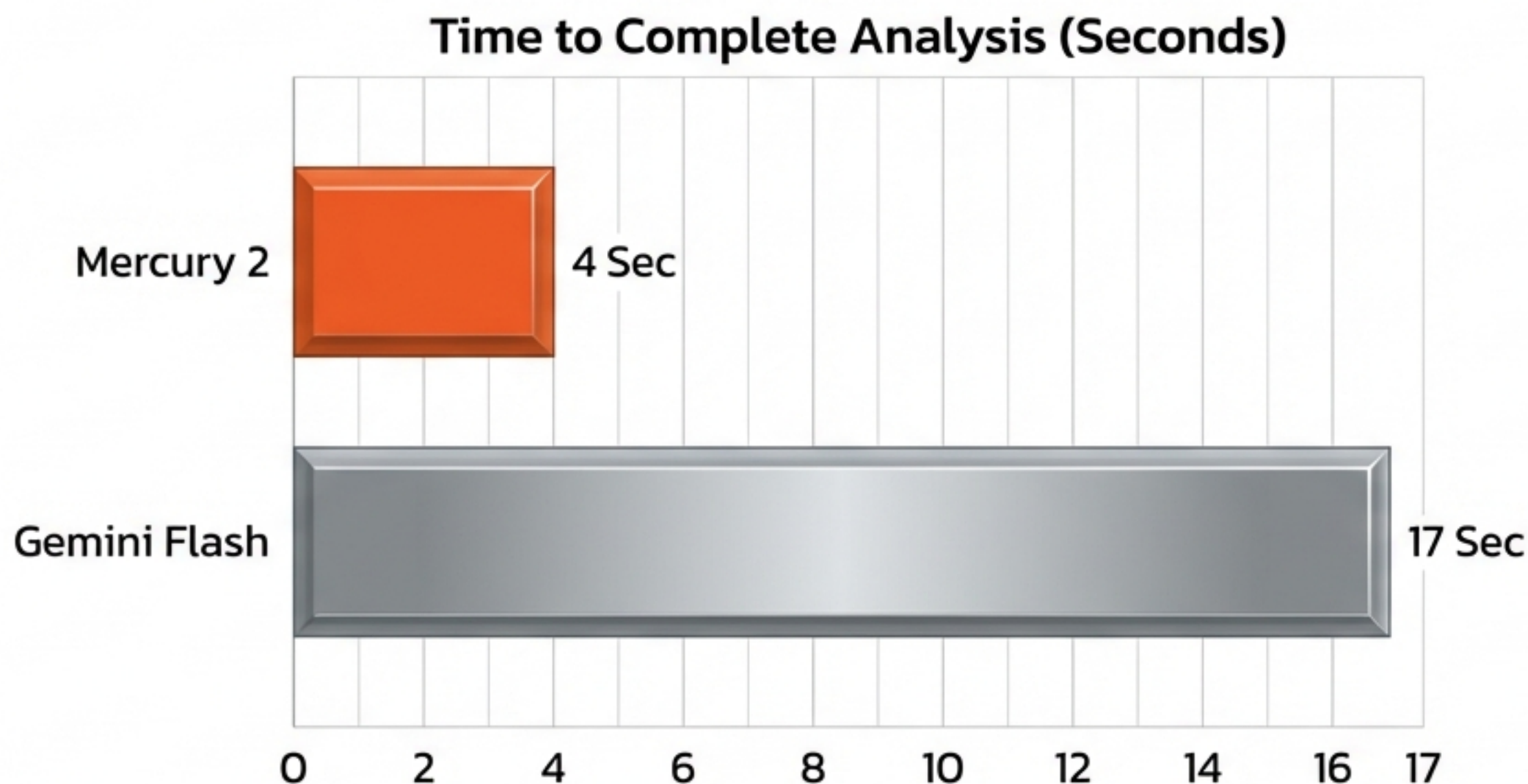


Almost Instant Response 🚿

การตอบสนองที่รวดเร็วระดับ Millisecond ทำให้บทสนทนาลื่นไหลเป็นธรรมชาติ
ไม่มีการรอโหลด (Awkward Pauses) เหมาะสำหรับระบบ Call Center อัตโนมัติ ⚡

Use Case: RAG และการวิเคราะห์เอกสารซับซ้อน

Scenario: การดึงข้อมูลราคาซื้อขายจากเอกสารสัญญา (Acquisition Agreement)



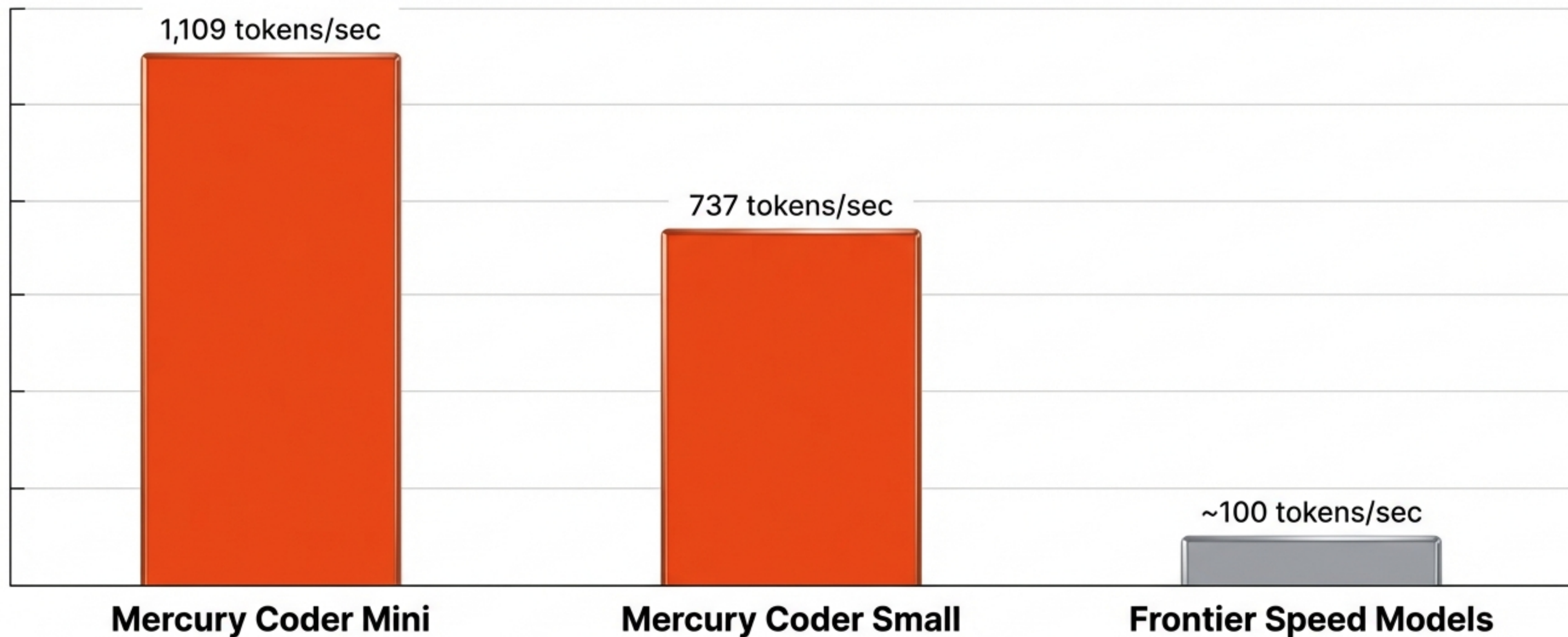
Mercury 2: 4 วินาที
(รวมการค้นหาและสรุปผล)

Gemini Flash: 17 วินาที

- สรุป: เร็วกว่า 4 เท่าในงาน Agentic workflow ที่ต้องมีการเรียกใช้เครื่องมือหลายขั้นตอน

ประสิทธิภาพที่พิสูจน์ได้ (Benchmarks)

เร็วกว่า Frontier Models ที่ปรับแต่งมาเพื่อความเร็ว
(Speed-optimized) ถึง 10 เท่า บนชิป NVIDIA H100



ความคุ้มค่าและสเปกสำหรับ 'งานหนัก' (Workhorse Model)

0.75

\$0.75 ต่อ 1 ล้าน Output Tokens

128k

Context Window (Tokens)

ออกแบบมาเพื่อ Agentic Applications
ที่ต้องทำงานตลอดเวลา ไม่ใช่แค่ Chatbot ทั่วไป

Mercury 2 เหมาะกับใคร?



Well-defined Tasks:
งานที่มีขอบเขตชัดเจนและ
ต้องการความเร็วสูง



Agentic Workflows:
ระบบที่ต้องมีการใช้ Tool
และ Logic หลายขั้นตอน



Real-time Applications:
ระบบโต้ตอบที่ Latency
เป็นเรื่องคอขาดบาดตาย

บทสรุป: อนาคตของ Diffusion LLM



“Diffusion LLM เป็นเทคโนโลยีใหม่ที่กำลังไล่ตาม
Auto-regressive มาอย่างรวดเร็ว...
นี้อาจเป็นมาตรฐานใหม่ของ AI ในอนาคต”